

HDOC Day-39 Assignment

Question 1: Weighted Average of User-Entered Value-Weight Pairs

1. Problem Understanding

The program reads count and the value-weight pairs from the user. Since the required calculation function is variadic, `weightedAverage(int count, ...)` receives count followed by separate value and weight arguments. The program supports user input for 1 to 10 pairs and calls the same variadic function with the entered values.

Note: Standard C cannot expand a runtime array automatically into variadic (...) arguments. Therefore, the program reads the pairs into variable-length arrays and uses a switch statement only to pass the entered elements as separate arguments to the genuine variadic function.

2. Standard Test Cases

S.No.	All Inputs	Expected Output
1	count = 3 Values: 80, 75, 90 Weights: 0.3, 0.2, 0.5	Weighted Average = 84.00
2	count = 2 Values: 10, 20 Weights: 1, 3	Weighted Average = 17.50
3	count = 4 Values: 5, 10, 15, 20 Weights: 1, 1, 1, 1	Weighted Average = 12.50

3. Algorithm

1. Start.
2. Read count from the user (1 to 10).
3. Create value and weight arrays of size count.
4. Use a loop to read each value and its corresponding weight.
5. Check that every weight is non-negative.
6. According to count, call `weightedAverage(count, ...)` and pass each value-weight pair as separate arguments.
7. In `weightedAverage()`, use `va_list` to read count pairs and calculate the weighted sum and total weight.
8. If total weight is zero, return an error value.
9. Otherwise return weighted sum / total weight.
10. Display the weighted average and stop.

4. C Program

```
#include<stdio.h>
#include<stdarg.h>
double weightedAverage(int count, ...)
{
    va_list args;
```

```

int i;
double value, weight;
double sum = 0, totalWeight = 0;
va_start(args, count);
for(i = 0; i < count; i++)
{
    value = va_arg(args, double);
    weight = va_arg(args, double);
    sum = sum + value * weight;
    totalWeight = totalWeight + weight;
}
va_end(args);
if(totalWeight == 0)
    return -1;
return sum / totalWeight;
}
int main()
{
    int count, i;
    double answer;
    printf("Enter count (1 to 10): ");
    scanf("%d", &count);
    if(count < 1 || count > 10)
    {
        printf("Invalid count.\n");
        return 0;
    }
    double value[count];
    double weight[count];
    for(i = 0; i < count; i++)
    {
        printf("Enter value %d: ", i + 1);
        scanf("%lf", &value[i]);
        printf("Enter weight %d: ", i + 1);
        scanf("%lf", &weight[i]);
        if(weight[i] < 0)
        {
            printf("Weight cannot be negative.\n");
            return 0;
        }
    }
    switch(count)
    {
        case 1:
            answer = weightedAverage(1, value[0], weight[0]);
            break;
        case 2:

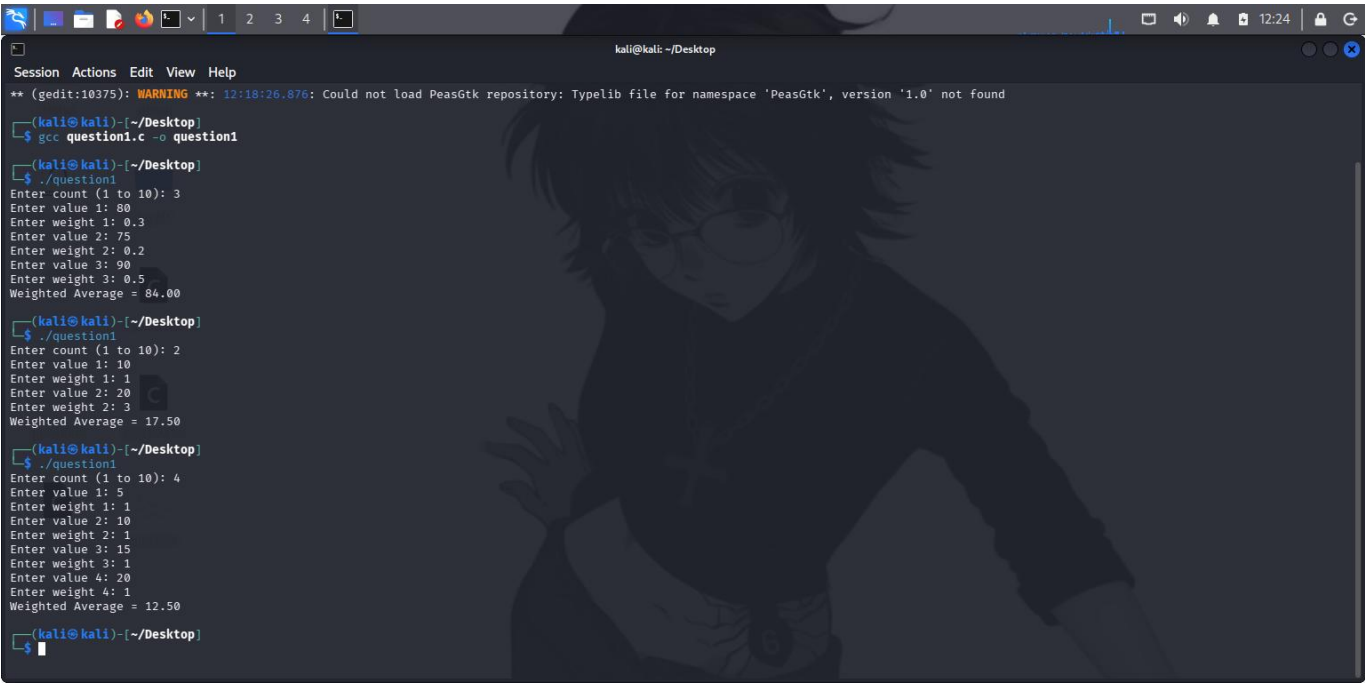
```

```

        answer = weightedAverage(2, value[0], weight[0], value[1], weight[1]);
        break;
    case 3:
        answer = weightedAverage(3, value[0], weight[0], value[1], weight[1], value[2], weight[2]);
        break;
    case 4:
        answer = weightedAverage(4, value[0], weight[0], value[1], weight[1], value[2], weight[2],
value[3], weight[3]);
        break;
    case 5:
        answer = weightedAverage(5, value[0], weight[0], value[1], weight[1], value[2], weight[2],
value[3], weight[3], value[4], weight[4]);
        break;
    case 6:
        answer = weightedAverage(6, value[0], weight[0], value[1], weight[1], value[2], weight[2],
value[3], weight[3], value[4], weight[4], value[5], weight[5]);
        break;
    case 7:
        answer = weightedAverage(7, value[0], weight[0], value[1], weight[1], value[2], weight[2],
value[3], weight[3], value[4], weight[4], value[5], weight[5], value[6], weight[6]);
        break;
    case 8:
        answer = weightedAverage(8, value[0], weight[0], value[1], weight[1], value[2], weight[2],
value[3], weight[3], value[4], weight[4], value[5], weight[5], value[6], weight[6], value[7], weight[7]);
        break;
    case 9:
        answer = weightedAverage(9, value[0], weight[0], value[1], weight[1], value[2], weight[2],
value[3], weight[3], value[4], weight[4], value[5], weight[5], value[6], weight[6], value[7], weight[7],
value[8], weight[8]);
        break;
    case 10:
        answer = weightedAverage(10, value[0], weight[0], value[1], weight[1], value[2], weight[2],
value[3], weight[3], value[4], weight[4], value[5], weight[5], value[6], weight[6], value[7], weight[7],
value[8], weight[8], value[9], weight[9]);
        break;
    }
    if(answer == -1)
        printf("Total weight cannot be zero.\n");
    else
        printf("Weighted Average = %.2lf\n", answer);
    return 0;
}

```

5. Compilation and Execution Evidence



```
Session Actions Edit View Help
** (gedit:10375): WARNING **: 12:18:26.876: Could not load PeasGtk repository: Typelib file for namespace 'PeasGtk', version '1.0' not found

(kali@kali)-[~/Desktop]
$ gcc question1.c -o question1

(kali@kali)-[~/Desktop]
$ ./question1
Enter count (1 to 10): 3
Enter value 1: 80
Enter weight 1: 0.3
Enter value 2: 75
Enter weight 2: 0.2
Enter value 3: 90
Enter weight 3: 0.5
Weighted Average = 84.00

(kali@kali)-[~/Desktop]
$ ./question1
Enter count (1 to 10): 2
Enter value 1: 10
Enter weight 1: 1
Enter value 2: 20
Enter weight 2: 3
Weighted Average = 17.50

(kali@kali)-[~/Desktop]
$ ./question1
Enter count (1 to 10): 4
Enter value 1: 5
Enter weight 1: 1
Enter value 2: 10
Enter weight 2: 1
Enter value 3: 15
Enter weight 3: 1
Enter value 4: 20
Enter weight 4: 1
Weighted Average = 12.50

(kali@kali)-[~/Desktop]
```

6. Verification of All Test Cases

Test	Input Summary	Actual Output	Result
1	3 pairs: (80,0.3), (75,0.2), (90,0.5)	84.00	Pass
2	2 pairs: (10,1), (20,3)	17.50	Pass
3	4 pairs: (5,1), (10,1), (15,1), (20,1)	12.50	Pass

Question 2: Integer Statistics from User Input

1. Problem Understanding

The program asks the user how many entries will be provided and then reads each entry as text. Only entries that represent valid integers are accepted. Invalid entries are ignored. The valid integers are sorted and the program displays their count, minimum, maximum, sum, and average.

2. Standard Test Cases

S.No.	All Inputs	Expected Output
1	n = 7 12, 5, abc, 8, 12, 20, 3	Valid: 6 Sorted: 3 5 8 12 12 20 Min=3, Max=20, Sum=60, Avg=10.00
2	n = 5 10, x, -2, 4, 8	Valid: 4 Sorted: -2 4 8 10 Min=-2, Max=10, Sum=20, Avg=5.00
3	n = 3 a, b, c	No valid integers

3. Algorithm

1. Start.
2. Read the number of entries n.
3. Read each entry as a string.
4. Check whether the complete entry is a valid integer.
5. Store only valid integers in an array.
6. If no valid integer exists, display a message and stop.
7. Sort the valid integers in ascending order.
8. Find minimum, maximum, sum, and average.
9. Display the valid count, sorted integers, minimum, maximum, sum, and average.
10. Stop.

4. C Program

```
#include<stdio.h>
#include<stdlib.h>

int main()
{
    int n, i, j, count = 0, temp;
    long sum = 0;
    char input[50], *end;

    printf("Enter number of entries: ");
    scanf("%d", &n);

    if(n <= 0)
```

```
{
    printf("Number of entries must be positive.\n");
    return 0;
}
```

```
int *a = (int *)malloc(n * sizeof(int));
```

```
for(i = 0; i < n; i++)
{
    long num;
    printf("Enter entry %d: ", i + 1);
    scanf("%49s", input);
```

```

    num = strtol(input, &end, 10);
    if(*input != '\0' && *end == '\0')
    {
        a[count] = (int)num;
        count++;
    }
}
```

```
if(count == 0)
{
    printf("No valid integers.\n");
    free(a);
    return 0;
}
```

```
for(i = 0; i < count - 1; i++)
{
    for(j = 0; j < count - i - 1; j++)
    {
        if(a[j] > a[j + 1])
        {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
    }
}
```

```
for(i = 0; i < count; i++)
    sum = sum + a[i];
```

```
printf("Valid integers = %d\n", count);
printf("Sorted integers: ");
for(i = 0; i < count; i++)
```

```

printf("%d ", a[i]);

printf("\nMinimum = %d\n", a[0]);
printf("Maximum = %d\n", a[count - 1]);
printf("Sum = %ld\n", sum);
printf("Average = %.2lf\n", (double)sum / count);

free(a);
return 0;
}

```

5. Compilation and Execution Evidence

```

kali@kali: ~/Desktop
Session Actions Edit View Help
(kali@kali)-[~/Desktop]
$ ./question2
Enter number of entries: 7
Enter entry 1: 12
Enter entry 2: 5
Enter entry 3: abc
Enter entry 4: 8
Enter entry 5: 12
Enter entry 6: 20
Enter entry 7: 3
Valid integers = 6
Sorted integers: 3 5 8 12 12 20
Minimum = 3
Maximum = 20
Sum = 60
Average = 10.00

(kali@kali)-[~/Desktop]
$ ./question2
Enter number of entries: 5
Enter entry 1: 10
Enter entry 2: x
Enter entry 3: -2
Enter entry 4: 4
Enter entry 5: 8
Valid integers = 4
Sorted integers: -2 4 8 10
Minimum = -2
Maximum = 10
Sum = 20
Average = 5.00

(kali@kali)-[~/Desktop]
$ ./question2
Enter number of entries: 3
Enter entry 1: a
Enter entry 2: b
Enter entry 3: c
No valid integers.

```

6. Verification of All Test Cases

Test	Input Summary	Actual Output	Result
1	12, 5, abc, 8, 12, 20, 3	6 valid; min 3; max 20; sum 60; avg 10.00	Pass
2	10, x, -2, 4, 8	4 valid; min -2; max 10; sum 20; avg 5.00	Pass
3	a, b, c	No valid integers	Pass